Graph Preconditioning for High-Dimensional Fixed Effects Regression

Alexander Fischer[1] and Kristof Schröder[2]

Draft: June 2026

Abstract. The Method of Alternating Projections (MAP) is the canonical algorithm for estimating high-dimensional fixed-effect regressions. It is fast when absorbed factors are well connected, but converges slowly on sparse or nearly nested fixed-effect graphs, such as matched employer-employee panels where worker-firm mobility links separate worker and firm effects. The convergence behavior of MAP depends on the mobility pattern linking workers to firms, yet MAP only indirectly makes use of this information by iterating over one fixed effect at a time. That mobility pattern is, however, directly encoded in the matrix of worker-firm match counts, which together with the worker and firm count diagonals forms a graph Laplacian after a sign flip and admits sparse approximate Cholesky factorization. We propose a graph-preconditioned Krylov solver whose reusable preconditioner is built from small, local factor-pair subproblems - worker-firm, worker-year, and so on - that use the graph directly. Benchmarks show MAP remains fastest on dense designs, while graph preconditioning reduces runtime on sparse worker-firm graphs and under strong sorting.

Keywords: high-dimensional fixed effects; alternating projections; preconditioning; matched employer-employee data; computational econometrics

JEL codes: C55; C63; C81; C87; J31

1. Introduction

Fixed-effect regressions are ubiquitous in applied econometrics: according to Goldsmith-Pinkham (2026), roughly half of published research in top economics and finance journals mentions "fixed effects". They appear throughout all fields of applied economics: labor economists use worker and firm fixed effects to separate worker heterogeneity from firm wage premia; health economists study physician practice styles with individual-physician and region fixed effects in mover designs; and education researchers study models with school, student, teacher, or student-teacher fixed effects.

The standard computational starting point for estimating these regressions efficiently is the Frisch-Waugh-Lovell (FWL) theorem (Frisch and Waugh 1933; Lovell 1963). FWL reduces the fixed-effect estimation problem to "residualizing" the outcome and every regressor of interest against the fixed effects, and then to running a low-dimensional regression on the residualized variables.

The workhorse method for these fixed-effect residualizations is the Method of Alternating Projections (MAP), also known as iterative demeaning or the "Zig-Zag" algorithm (Guimaraes and Portugal 2010; Gaure 2013). Most leading software implementations of fixed-effect regression, such as `reghdfe` in Stata

[1]trivago

[2]appliedAI Institute for Europe gGmbH

(Correia 2014; 2017), `fixest` (Bergé et al. 2026) in R, or PyFixest in Python (PyFixest Developers 2026), use MAP or MAP-based variants with acceleration.

Because the AKM worker-firm model is among the most prominent applications of high-dimensional fixed effects, we use a worker-firm-year panel based on Abowd et al. (1999) as the running example in this paper. The method of alternating projections cycles through the fixed-effect dimensions one at a time: it first subtracts worker means, then firm means, then year means, and repeats until convergence. The coupling between fixed effects is never used directly; the cross-fixed effects information is transmitted only indirectly through the residual that each step hands to the next. How fast MAP converges therefore depends on how quickly information about this coupling can propagate from one update to the next.

Fixed effects and their coupling form a graph structure. In a worker-firm panel, movers create paths between firms, while stayers add observations without connecting firms to one another. When this graph is sparse or poorly connected, some fixed-effect directions are nearly collinear, and MAP needs many iterations to propagate information across it before converging. The same graph features that determine whether worker and firm effects are identified also govern MAP convergence. These features include worker mobility, sorting, and segmentation into disconnected labor markets with thin bridges, such as public- and private-sector employment when workers only rarely move between the two sectors.

The graph structure of the fixed effects can be algebraically encoded in the off-diagonal blocks of the fixed-effect Gramian (Correia 2017). These off-diagonal blocks record co-occurrences among the fixed effects: which workers work at which firms, which physicians practice in which regions, or which families move across counties.

In this paper, we propose a new preconditioner that directly encodes the co-occurrence graph. A preconditioner is a cheap approximation to the system being solved; supplied to an iterative solver, it reduces the number of iterations without changing the solution. We build ours from local factor-pair subproblems that use the graph structure of the Gramian: for example, a worker-firm subproblem incorporates the observed links between workers and firms. A preconditioner is only useful if it approximates the inverse of the co-occurrence Gramian at low cost. We obtain such an approximation by exploiting the fact that, after a sign change, each factor-pair block is a graph Laplacian, which admits sparse approximate inverses following ideas from the Laplacian-solver literature (Spielman and Teng 2014; Gao et al. 2025). The preconditioned system is then solved with a Krylov solver, an iterative method for large linear systems.[3] In a range of benchmarks against mature implementations of the method of alternating projections, we find that on sparse, poorly connected graphs (the regime where MAP convergence deteriorates) the graph-preconditioned solver lowers runtime, while on dense, well-connected graphs the preconditioner setup cost does not amortize and MAP should remain the natural default.

The rest of the paper is organized as follows. Section 2 sets up the fixed-effect absorption problem, and Section 3 introduces the AKM model as our running example. Section 4 develops the graph structure of the fixed-effect Gramian, and Section 5 connects this structure to the convergence behavior of MAP. Section

[3]The Julia implementation of fixed-effect regression, `FixedEffectModels.jl` (FixedEffectModels.jl Developers 2026), uses the same Krylov solver as we do - LSMR (Fong and Saunders 2011) - but only with diagonal preconditioning, which ignores the off-diagonal co-occurrence structure entirely; our contribution is the preconditioner, not the use of LSMR for the outer iteration.

6 builds up the factor-pair Schwarz preconditioner, starting from a general discussion of preconditioning and culminating in the construction of the graph-based preconditioner. Section 7 reports benchmarks on runtime, memory, and numerical equivalence; Section 8 describes the software through which the new algorithm is available; and Section 9 concludes.

2. Absorbing Fixed Effects[4]

We focus on the linear model

$$y = X\beta + D\alpha + \varepsilon, \tag{1}$$

where X contains the regressors of interest, D is the fixed-effect design matrix, and α collects the fixed-effect coefficients. In high-dimensional applications D may have hundreds of thousands or millions of columns, and forming or inverting the full system in $[X \quad D]$ might prove computationally infeasible. Fortunately, via the Frisch-Waugh-Lovell (FWL) theorem, we can compute $\hat{\beta}$ without ever forming $[X \quad D]$ or inverting its cross product.

FWL reduces the computation of $\hat{\beta}$ to two steps. In step one, we residualize the outcome and each covariate against the fixed effects: we regress y on D and keep the residual $\tilde{y}$, and we do the same for every column of X. Denoting M_D as the linear operator that sends a variable to this residual, so that $\tilde{y} = M_D y$ and $\tilde{X} = M_D X$, the coefficient of interest is recovered by regressing the residualized outcome on the residualized covariates,

$$\tilde{y} = M_D y, \quad \tilde{X} = M_D X, \quad \hat{\beta} = \left(\tilde{X}' W \tilde{X}\right)^{-1} \tilde{X}' W \tilde{y}, \tag{2}$$

where W is a diagonal matrix of weights. With one covariate, the procedure consists of three regressions: y on D, x on D, and $\tilde{y}$ on $\tilde{x}$. FWL implies that the slope in the last regression equals the coefficient on x in the full regression of y on x and D.

The residualization step is the weighted least-squares projection of the outcome and each covariate in X onto the column space of the fixed-effect dummy matrix D. For a right-hand side μ, either y or one column of X, we solve

$$\hat{\alpha}_\mu = \arg\min_\alpha \| \, D\alpha - \mu \, \|_W^2, \tag{3}$$

and the residual is $\tilde{\mu} = \mu - D\hat{\alpha}_\mu$. The first-order condition for Equation 3,

$$D' W \left(D\hat{\alpha}_\mu - \mu\right) = 0, \quad \text{equivalently} \quad G\hat{\alpha}_\mu = D' W \mu, \quad G = D' W D, \tag{4}$$

determines $\hat{\alpha}_\mu$. Each FWL residualization uses the same coefficient matrix G; only the right-hand side changes as we move from the outcome to the covariates. The cost of residualization therefore depends on the structure of the Gramian G. In the next section, we illustrate this structure with the AKM worker-firm model and explain why fixed-effect designs have a direct graph interpretation. Sections 5–6 then return to the algorithms and show how the structure of the Gramian and its associated graph governs MAP convergence, and explain how we use it to construct the factor-pair Schwarz preconditioner.

[4]Researchers employ several names for this operation: "absorbing fixed effects", "demeaning", "residualizing", or applying the "within transformation". We use these terms interchangeably throughout.

3. A Running Example: The AKM Model

The AKM model of Abowd et al. (1999) introduces the worker-firm setting used throughout the paper. It separates persistent worker heterogeneity from firm wage premia using workers who move across firms. We write the AKM regression equation as

$$y_{it} = \alpha_i + \psi_{J(i,t)} + \varphi_t + x_{it}'\beta + \varepsilon_{it}, \tag{5}$$

where α_i is a worker fixed effect, $\psi_{J(i,t)}$ is the fixed effect for the firm employing worker i at time t, and φ_t is a time fixed effect.

The AKM specification has a natural graph representation. Workers and firms are nodes in a bipartite graph, and each employment spell contributes an edge. Worker moves induce a firm-to-firm graph, in which two firms are connected when at least one worker is observed at both firms. Although stayers - workers who never change their employer - add observations to existing worker-firm links, they do not create bridges between firms. Year effects enter as a third, low-dimensional factor observed on the same worker-firm records. Figure 1 contrasts a well-connected mobility graph with one that fragments under strong sorting.

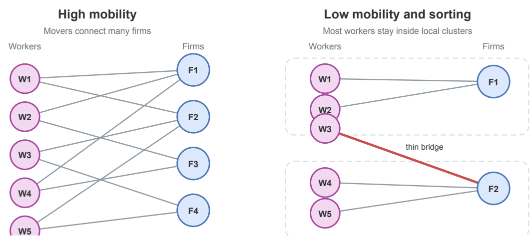

Figure 1: Worker-firm graph connectivity. When mobility is high, many paths connect firms. With low mobility and strong sorting, the graph breaks into nearly separate clusters joined only by narrow bridges.

These mobility links matter both for identification and, as we will argue later, for computation. In AKM, worker and firm fixed effects are separately identified through movers, who provide the comparisons that distinguish worker heterogeneity from firm wage premia. A worker observed at only one firm provides no such comparison: a high wage could reflect an unusually productive worker, a high-wage firm, or both. Without worker moves, these components are not separately identified. Movers observed across firms with different wage profiles help attribute wage variation to worker effects or firm premia. If a worker earns high wages across several firms, the comparison points toward a worker effect; if many different workers earn higher wages at the same firm, it points toward a firm premium. The more such cross-firm comparisons the data contain, especially across otherwise different firms, the easier it is to identify worker and firm premia. In the graph, these moves are exactly the edges that connect firms; additional moves of workers across firms add worker-firm links to the graph.

4. The Graph Structure of the Gramian

The bipartite graph of worker and firm connections introduced in Section 3 has an algebraic representation in the block structure of the Gramian $G = D'WD$ (Correia 2017). Suppose that the columns of D are ordered as worker levels, firm levels, and year levels. Then

$$G = \begin{pmatrix} G_{WW} & C_{WF} & C_{WY} \\ C_{(WF)'} & G_{FF} & C_{FY} \\ C_{(WY)'} & C_{(FY)'} & G_{YY} \end{pmatrix}. \tag{6}$$

The diagonal blocks G_{WW}, G_{FF}, and G_{YY} contain weighted counts for workers, firms, and years. An observation belongs to one level of each factor, so these blocks are diagonal; solving them requires only division by group counts.

The off-diagonal blocks are cross-tabulations: the worker-firm block C_{WF} records how often worker i is observed at firm j, and the worker-year and firm-year blocks have analogous interpretations.

As a small example, we construct a worker-firm panel and populate its Gramian. For simplicity, we ignore any regression weights and set $W = I$.

Obs.	Worker	Firm	Year	y
1	W_1	F_1	Y_1	3.2
2	W_1	F_2	Y_2	4.1
3	W_2	F_1	Y_1	2.8
4	W_2	F_1	Y_2	3.9
5	W_3	F_2	Y_1	5.0
6	W_3	F_2	Y_2	4.5

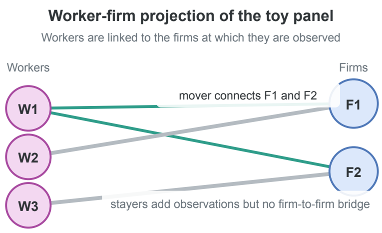

Figure 2: Worker-firm projection of the example panel. Worker W_1 is a mover; workers W_2 and W_3 are stayers.

Figure 2 plots the worker-firm projection of this panel. Worker W_1 has employment spells both in F_1 and F_2 and creates a link between the two firms; in AKM terms, W_1 is a mover. Worker W_2 stays at F_1 for two periods, and W_3 stays at F_2 for two periods. Both are stayers.

The diagonal blocks are count matrices. In this example, each worker is observed twice, each firm three times, and each year three times, so

$$G_{WW} = \begin{pmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 2 \end{pmatrix}, \quad G_{FF} = \begin{pmatrix} 3 & 0 \\ 0 & 3 \end{pmatrix}, \quad G_{YY} = \begin{pmatrix} 3 & 0 \\ 0 & 3 \end{pmatrix}. \tag{7}$$

The off-diagonal blocks are cross-tabulations between factors. The worker-firm block is

$$C_{WF} = \begin{pmatrix} 1 & 1 \\ 2 & 0 \\ 0 & 2 \end{pmatrix}. \tag{8}$$

The first row indicates that worker W_1 appears once at firm F_1 and once at firm F_2. Workers W_2 and W_3 are stayers, appearing twice at F_1 and F_2, respectively. The worker-year block is

$$C_{WY} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \\ 1 & 1 \end{pmatrix}. \tag{9}$$

Each worker is observed once in each year. The firm-year block is

$$C_{FY} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}. \tag{10}$$

Firm F_1 appears twice in year Y_1 and once in year Y_2; firm F_2 has the opposite pattern. Combining the diagonal count blocks and the off-diagonal cross-tabulation blocks yields the full Gramian.

With column order $(W_1, W_2, W_3, F_1, F_2, Y_1, Y_2)$, the full Gramian is

$$G = \left(\begin{array}{ccc|cc|cc} 2 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 2 & 0 & 2 & 0 & 1 & 1 \\ 0 & 0 & 2 & 0 & 2 & 1 & 1 \\ \hline 1 & 2 & 0 & 3 & 0 & 2 & 1 \\ 1 & 0 & 2 & 0 & 3 & 1 & 2 \\ \hline 1 & 1 & 1 & 2 & 1 & 3 & 0 \\ 1 & 1 & 1 & 1 & 2 & 0 & 3 \end{array}\right). \tag{11}$$

The worker-firm submatrix stores the bipartite graph algebraically. The diagonal entries are worker and firm counts, and the entries of C_{WF} are edge multiplicities between workers and firms. After flipping the sign of C_{WF}, this submatrix is a graph Laplacian: its off-diagonal entries are non-positive, and every row sums to zero because each diagonal count cancels the off-diagonal spell counts in the same row. For a worker, the diagonal entry is the number of that worker's employment spells, while the off-diagonal entries count how those spells are distributed across firms; firm rows have the analogous interpretation with spell counts summed over workers.

$$L_{WF} = \left(\begin{array}{ccc|cc} 2 & 0 & 0 & -1 & -1 \\ 0 & 2 & 0 & -2 & 0 \\ 0 & 0 & 2 & 0 & -2 \\ \hline -1 & -2 & 0 & 3 & 0 \\ -1 & 0 & -2 & 0 & 3 \end{array}\right). \tag{12}$$

The same Laplacian construction applies to any pair of fixed effects, and the preconditioner of Section 6 builds on these pairwise Laplacians. Before turning to it, however, we introduce the method of alternating projections, which avoids forming the full Gramian G by working only on the diagonal worker, firm, and year blocks.

5. Alternating Projections and Graph Connectivity

The workhorse algorithm for multi-way fixed effects is the Method of Alternating Projections (MAP), also referred to as iterative demeaning or the "zig-zag" algorithm (Guimaraes and Portugal 2010; Gaure 2013). Many packages employ MAP or its variants, frequently combined with accelerations (Berge 2018; Correia 2017), such as the Irons-Tuck extrapolation used by `fixest` (Irons and Tuck 1969; Bergé et al. 2026). Sections 3 and 4 introduced the fixed-effect graph and its algebra; this section turns to MAP itself and shows how that graph geometry governs its convergence rate.

MAP solves the FWL residualization problem by iterating over one fixed effect at a time. In the worker-firm-year model, MAP first subtracts worker means from the current residual, then firm means from the updated residual, then year means, and so on until convergence.

We write the FWL normal equations (see Equation 4) in block form with $D = [D_W \quad D_F \quad D_Y]$ as

$$\begin{pmatrix} G_{WW} & C_{WF} & C_{WY} \\ C_{(WF)'} & G_{FF} & C_{FY} \\ C_{(WY)'} & C_{(FY)'} & G_{YY} \end{pmatrix} \begin{pmatrix} \alpha_W \\ \alpha_F \\ \alpha_Y \end{pmatrix} = \begin{pmatrix} D_{W'} W \mu \\ D_{F'} W \mu \\ D_{Y'} W \mu \end{pmatrix}. \tag{13}$$

Equivalently,

$$G_{WW}\alpha_W + C_{WF}\alpha_F + C_{WY}\alpha_Y = D_{W'} W \mu, \tag{14}$$

$$C_{(WF)'}\alpha_W + G_{FF}\alpha_F + C_{FY}\alpha_Y = D_{F'} W \mu, \tag{15}$$

$$C_{(WY)'}\alpha_W + C_{(FY)'}\alpha_F + G_{YY}\alpha_Y = D_{Y'} W \mu. \tag{16}$$

Each equation can be rearranged to express one block of effects conditional on the others. Using $C_{WF} = D_{W'} W D_F$ and $C_{WY} = D_{W'} W D_Y$ to factor $D_{W'} W$ out of the right-hand side, the first equation becomes

$$G_{WW}\alpha_W = D_{W'} W(\mu - D_F\alpha_F - D_Y\alpha_Y). \tag{17}$$

Because G_{WW} is a diagonal matrix whose entries are workers' total observation weights, solving for α_W divides each worker's weighted partial residual by its total observation weight. We apply the same rearrangement to the second equation to obtain the firm equation

$$G_{FF}\alpha_F = D_{F'} W(\mu - D_W\alpha_W - D_Y\alpha_Y), \tag{18}$$

and the year equation is analogous. Because G_{WW}, G_{FF}, and G_{YY} are all diagonal, each of the three equations is solved by computing a weighted group mean in a single pass over observations.

MAP uses this diagonal structure iteratively. Holding the other effects fixed, it updates the worker effects from the current partial residual, then repeats the same step for firms and years. Each sweep therefore cycles through the fixed-effect dimensions, subtracting the weighted group mean of the current partial residual for the factor being updated.

The cross-tabulation blocks C_{WF}, C_{WY}, and C_{FY} enter the algorithm only indirectly. For instance, the worker update is computed from the partial residual $\mu - D_F\alpha_F - D_Y\alpha_Y$, while the firm update is computed from $\mu - D_W\alpha_W - D_Y\alpha_Y$. Worker-firm, worker-year, and firm-year links are thus not solved as coupled subproblems; their effect propagates through the residual that one block update transmits to the next.

This strategy is effective when the graph is well connected. In a high-mobility worker-firm panel, many workers move across firms, so that worker and firm effects can be compared through many overlapping employment histories. A high wage at one firm can then be related to wages earned by the same workers at other firms, and a worker update rapidly alters the information available to the next firm update, and vice versa.

When mobility is sparse, sorting is strong, or one factor is nearly nested in another, MAP slows down. The information that separates worker effects from firm effects travels through movers, the edges of the worker-firm graph, and MAP picks it up only indirectly, through repeated residual updates. When those edges are few or bunched into narrow regions of the graph, each further iteration carries little fresh information. MAP still converges, but it may need many sweeps; each sweep is cheap because the diagonal block solves reduce to one-pass group means.

The same graph perspective gives a simple diagnostic for MAP difficulty in a fixed-effect structure. For any pair of fixed effects (q, r), we form the normalized cross-tabulation $H_{qr} = G_{qq}^{-\frac{1}{2}} C_{qr} G_{rr}^{-\frac{1}{2}}$ and let $\rho_{qr} = \sigma_2(H_{qr})^2$ be the square of its largest nontrivial singular value, equivalently the largest nontrivial eigenvalue of $H_{(qr)'} H_{qr}$. The largest singular value of H_{qr} equals one on every connected component and carries no information about connectivity, so we discard it. We report the spectral gap $1 - \rho_{qr}$ in the benchmarks below. This gap measures how well connected the factor-pair graph is.[5] Gaps near zero signal sparse mobility or near-nesting, the settings in which MAP converges slowly; larger gaps indicate better connected factor-pair graphs. For the worker-firm example of Section 4, the gap is $\frac{1}{3}$.[6]

6. The Factor-Pair Schwarz Preconditioner

6.1. Preconditioners

The previous section attributed MAP's slow convergence to thin connections in the fixed-effect graph. A solver that receives this connectivity information directly should converge faster. MAP has no natural way to accept it: its update rule is fully determined by the list of absorbed factors, and the cross-tabulations enter only through the residual that one update passes to the next. Krylov solvers, by contrast, take an additional input, the preconditioner, and this input is where we place the graph structure. We therefore replace factor-by-factor demeaning via MAP with LSMR (Fong and Saunders 2011), an iterative least-squares algorithm that improves an initial guess through repeated residual corrections. The change of solver gains little by itself: a poorly conditioned Gramian G slows LSMR just as a poorly connected graph slows MAP. The core acceleration stems from building a good preconditioner, which we focus on in the remainder of this section.

How fast LSMR converges depends on the conditioning of G. When G is well conditioned, LSMR shrinks every component of the residual at a comparable rate. When G is poorly conditioned, LSMR removes some components after a few iterations, whereas components that correspond to weak links, sparse mobility, or near nesting in the fixed-effect graph shrink much more slowly; the iteration then spends most of its steps on these slow components. A preconditioner counteracts this imbalance: it changes the coordinates of the linear system so that slow and fast components decay at more similar rates, without changing the least-squares solution.

The ideal preconditioner is G^{-1}.[7] If $M^{-1} = G^{-1}$, then the preconditioned operator is

---

[5]When the graph has more than one connected component, we compute the gap on each component and report the smallest, together with its observation share.

[6]In that example, $G_{WW} = \operatorname{diag}(2,2,2)$, $G_{FF} = \operatorname{diag}(3,3)$, and $C_{WF} = \begin{pmatrix} 1 & 1 \\ 2 & 0 \\ 0 & 2 \end{pmatrix}$, so $H_{WF} = G_{WW}^{-\frac{1}{2}} C_{WF} G_{FF}^{-\frac{1}{2}} = \frac{1}{\sqrt{6}} \begin{pmatrix} 1 & 1 \\ 2 & 0 \\ 0 & 2 \end{pmatrix}$. The graph is connected, and $H_{(WF)'} H_{WF} = \frac{1}{6} \begin{pmatrix} 5 & 1 \\ 1 & 5 \end{pmatrix}$ has eigenvalues 1 and $\frac{2}{3}$. After dropping the unit eigenvalue, $\rho_{WF} = \frac{2}{3}$ and the gap is $\frac{1}{3}$.

[7]As in any model with several fixed effects, the level effects are pinned down only up to a normalization: we can add a constant to every worker effect and subtract it from every firm effect without changing the fitted values $D\alpha$, and likewise for years. The dummy-coded $G = D'WD$ and the smaller blocks inverted below are therefore not invertible until this

$$M^{-1}G = G^{-1}G = I. \tag{19}$$

In exact arithmetic, the Krylov iteration would then recover the solution after one correction, because the system has no slow directions left to remove. To form G^{-1} we would have to solve the fixed-effect normal equations themselves, so the identity case serves as a benchmark rather than an implementable preconditioner. A useful preconditioner must approximate enough of G^{-1} to remove the slow directions of the iteration, while its construction and repeated application must amortize over the iterations it saves.[8]

6.2. From the Block Inverse to the Diagonal Preconditioner

The block structure of G^{-1} shows which parts of the ideal inverse a feasible preconditioner should retain. For the worker-firm-year AKM model, the Gramian has the block form

$$G = \begin{pmatrix} G_{WW} & C_{WF} & C_{WY} \\ C_{(WF)'} & G_{FF} & C_{FY} \\ C_{(WY)'} & C_{(FY)'} & G_{YY} \end{pmatrix}, \tag{20}$$

with diagonal weighted-count blocks G_{WW}, G_{FF}, G_{YY} and off-diagonal cross-tabulations C_{WF}, C_{WY}, C_{FY}. Block inversion via the Schur complement shows how each off-diagonal block enters G^{-1}. For the two-factor block

$$G_2 = \begin{pmatrix} G_{WW} & C_{WF} \\ C_{(WF)'} & G_{FF} \end{pmatrix}, \tag{21}$$

it gives the explicit formula

$$G_2^{-1} = \begin{pmatrix} G_{WW}^{-1} + G_{WW}^{-1} C_{WF} S^{-1} C_{(WF)'} G_{WW}^{-1} & -G_{WW}^{-1} C_{WF} S^{-1} \\ -S^{-1} C_{(WF)'} G_{WW}^{-1} & S^{-1} \end{pmatrix}. \tag{22}$$

$$S = G_{FF} - C_{(WF)'} G_{WW}^{-1} C_{WF}. \tag{23}$$

The formula shows that every block of G_2^{-1} depends on C_{WF} only through the Schur complement S: the cross-tabulation enters through matrix products, never as its own inverse. G_{WW} is diagonal, so G_{WW}^{-1} is a division by weighted worker counts. The Schur complement S, by contrast, is the firm-side mobility system that remains after eliminating workers. At the scale of modern worker-firm register data, solving this system is expensive: exact factorization creates many additional nonzero entries, separate connected components require separate normalizations, and weak mobility makes the remaining directions slow to resolve. S^{-1} therefore carries almost all the cost of the solve. Block inversion via Schur complements generalizes to three factors: the closed form has more terms, but every block of G^{-1} still depends jointly on the cross-tabulations C_{WF}, C_{WY}, C_{FY}.

The coarsest approximation to G^{-1} keeps only the diagonal count inverses and drops the Schur-complement corrections,

$$M_{\text{diag}}^{-1} = \operatorname{diag}(G_{WW}^{-1}, G_{FF}^{-1}, G_{YY}^{-1}). \tag{24}$$

indeterminacy is removed – one normalization for the worker-firm pair, and a second once year effects are added, as in the Section 4 example. We adopt the standard normalization within each connected set and read every inverse below as that of the resulting system. The estimated effects depend on the normalization; the residualized outcome and regressors, which are all the regression uses, do not.

[8] LSMR never forms $M^{-1}G$ explicitly, nor G itself. The iteration requires only products with D and D' and applications of M^{-1}, supplied as linear operators.

This preconditioner is a single division by weighted level counts. It is the diagonal preconditioner used with LSMR in `FixedEffectModels.jl` (Fong and Saunders 2011; FixedEffectModels.jl Developers 2026). Diagonal scaling encodes how many observations a level carries, but not how that level connects to the rest of the labor market. Those counts can already be useful when employment is concentrated in a few large firms, such as Novo Nordisk in Denmark or Samsung in South Korea, because the size differences alone remove an important source of scale variation. What diagonal scaling does not record is whether workers at those firms link them broadly to other firms or remain concentrated in a narrow corner of the mobility graph.

6.3. The Factor-Pair Schwarz Approximation

Additive Schwarz preconditioning adds this missing pairwise connectivity without solving the full three-factor system (Xu 1992; Toselli and Widlund 2005). It splits the fixed-effect problem into smaller overlapping pair problems. In the AKM case, one problem contains workers and firms, another contains workers and years, and a third contains firms and years. The worker-firm problem moves residual information along observed employment links; the other two pair problems do the same for worker-year and firm-year links. We then combine the three pair corrections in the full coefficient space. The preconditioner therefore gives the Krylov iteration the main pairwise channels of the Gramian, while the outer iteration handles the remaining three-way coupling.

To make the local subproblems concrete, we first consider the worker-firm pair. Its local problem is exactly the two-factor block from the Schur calculation,

$$\begin{pmatrix} G_{WW} & C_{WF} \\ C_{(WF)'} & G_{FF} \end{pmatrix}. \tag{25}$$

Figure 3 places this worker-firm pair block beside the single diagonal block solved by a factor-level MAP update.

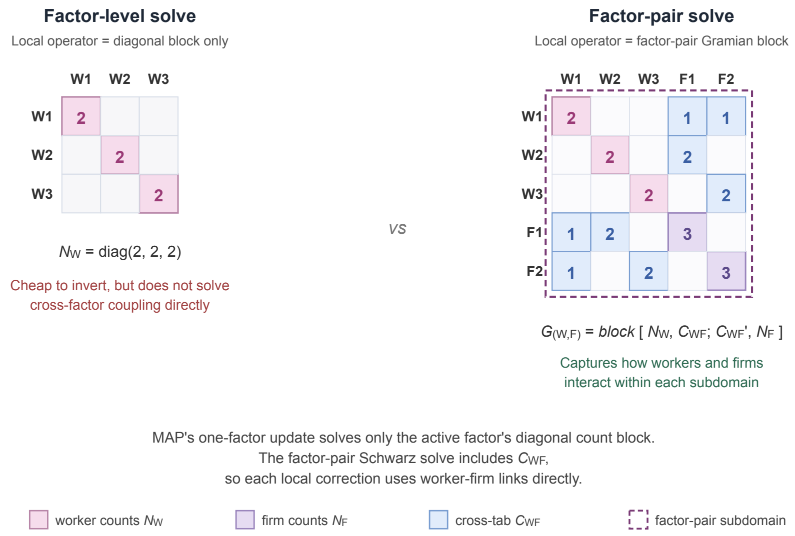

Figure 3: Local operator used by a factor-level MAP update (left) versus the factor-pair Schwarz solve (right), shown on the example worker-firm panel of Section 4. The factor-level block is the diagonal $G_{WW} = \mathrm{diag}(2, 2, 2)$. The factor-pair block adds the firm count block $G_{FF} = \mathrm{diag}(3, 3)$ and the cross-tabulation C_{WF} in its off-diagonal positions; the dashed outline marks the worker-firm subdomain on which the local Schwarz solve operates.

Its inverse carries C_{WF} through the Schur complement, so the local correction holds the worker-firm mobility geometry that diagonal scaling discards. To place this correction inside the three-factor problem, let R_{WF} select the worker and firm entries from the full coefficient vector $\alpha = [\alpha_W; \alpha_F; \alpha_Y]$; its transpose $R_{(WF)'}$ places the resulting correction back into the full vector. The diagonal matrix $\tilde{D}_{WF}$ contains the weights that split levels across the pair problems in which they appear; we call these entries partition-of-unity weights. The exact worker-firm contribution is

$$P_{WF}^{-1} = R_{(WF)'} \tilde{D}_{WF} \begin{pmatrix} G_{WW} & C_{WF} \\ C_{(WF)'} & G_{FF} \end{pmatrix}^{-1} \tilde{D}_{WF} R_{WF}. \tag{26}$$

The worker-year and firm-year contributions P_{WY}^{-1} and P_{FY}^{-1} are built the same way from G_{WW}, C_{WY}, G_{YY} and G_{FF}, C_{FY}, G_{YY}. With three factors each level appears in exactly two pair problems. Because the weights act on both sides of each pair contribution, we set them so that the squared weights on a shared level sum to one; a level in two pairs then carries $\frac{1}{\sqrt{2}}$. The exact factor-pair Schwarz preconditioner is the sum of these three contributions,

$$P^{-1} = P_{WF}^{-1} + P_{WY}^{-1} + P_{FY}^{-1}. \tag{27}$$

The three terms include the worker-firm, worker-year, and firm-year cross-tabulations separately. They do not solve the simultaneous worker-firm-year problem; that remaining coupling, together with the error introduced by splitting shared levels across pairs, is left to the outer LSMR iteration.

6.4. Approximate Pair Solves via Graph Laplacians

For P^{-1} to serve as the operator M^{-1} inside LSMR, each pair contribution must be computed without solving a large dense system. For factor pairs with few levels, we invert the pair block directly. In the example worker-firm panel of Section 4, the pair block has three worker levels and two firm levels; after the normalization of Section 6.1 removes the one free constant, the local solve is a 4×4 inversion. At the scale of modern worker-firm register data, however, a single pair can carry hundreds of thousands of levels per side, and direct inversion becomes the same kind of large linear-algebra problem the preconditioner is meant to avoid. We instead use the graph-Laplacian structure of the pair block.

For a worker-firm pair, the local Schwarz step solves the pair-Gramian system

$$\begin{pmatrix} G_{WW} & C_{WF} \\ C_{(WF)'} & G_{FF} \end{pmatrix} x = u, \tag{28}$$

where u is the weighted worker-firm part of the current Krylov residual. This block is not a graph Laplacian, because its off-diagonal entries C_{WF} are non-negative. A sign flip removes the obstacle. Let $T_{WF} = \operatorname{diag}(I_W, -I_F)$ flip the sign of the firm entries, so that $T_{WF}^2 = I$. Conjugation by T_{WF} gives

$$L_{WF} = T_{WF} \begin{pmatrix} G_{WW} & C_{WF} \\ C_{(WF)'} & G_{FF} \end{pmatrix} T_{WF} = \begin{pmatrix} G_{WW} & -C_{WF} \\ -C_{(WF)'} & G_{FF} \end{pmatrix}, \tag{29}$$

a weighted bipartite graph Laplacian: symmetric, with non-positive off-diagonals and zero row sums. Because $T_{WF}^2 = I$, the pair-Gramian solve follows from the Laplacian solve by the same flip on each side,

$$\begin{pmatrix} G_{WW} & C_{WF} \\ C_{(WF)'} & G_{FF} \end{pmatrix}^{-1} = T_{WF} L_{WF}^{-1} T_{WF}, \tag{30}$$

where both inverses are read as in Section 6.1: the solve fixes the free constant on each connected component by returning the zero-mean solution, and the residualized variables do not depend on this choice. A single Laplacian solve therefore yields the pair-Gramian solution: we flip the residual, apply L_{WF}^{-1}, and flip back.

For preconditioning, this local solve need not be exact. The outer LSMR iteration refines any error left by the preconditioner. We therefore approximate the Laplacian solve using sparse approximate Cholesky factorizations from the Laplacian-solver literature (Spielman and Teng 2014; Gao et al. 2025). An exact Cholesky factorization of the pair block creates fill-in: eliminating a worker inserts entries linking every pair of distinct firms that worker visited, and these entries accumulate as the elimination proceeds. In the worst case, the cost approaches the order k^3 operations and order k^2 memory of a dense factorization of a k-level system. The randomized approximate factorization avoids this growth on any graph: its cost is nearly linear in the number of observed worker-firm links, that is, linear up to logarithmic factors. After transforming this approximate Laplacian solve back to worker-firm coordinates, we write A_{WF} for the resulting approximate pair-Gramian solve.

The same construction yields A_{WY} and A_{FY} for the other two pairs. We substitute these approximate inverses into the Schwarz sum to obtain the implemented preconditioner,

$$M^{-1} = \sum_{(q,r)} R_{(qr)'} \tilde{D}_{qr} A_{qr} \tilde{D}_{qr} R_{qr}. \tag{31}$$

In the worker-firm-year case each factor receives a diagonal contribution from its two pair subdomains and each off-diagonal correction from the corresponding pair.

The approximate pair inverse introduces a second approximation, beyond the pairwise splitting. The exact P^{-1} already replaces the full three-factor inverse with pair solves; M^{-1} now applies each pair solve only approximately, through A_{qr}. Both choices shape the preconditioned search directions and nothing else: the fitted residuals are unchanged, and LSMR still solves the original fixed-effect least-squares problem to the requested tolerance.

6.5. Implementation Strategy

Figure 4 summarizes the full construction. We start with the absorbed factors and split the fixed-effect graph into overlapping factor pairs. For each pair, the local Gramian block is represented as a graph Laplacian after a sign flip; sparse approximate Cholesky solves then provide an approximate pair inverse, returned to Gramian coordinates. The pair corrections are combined with partition-of-unity weights to form the Schwarz preconditioner M^{-1}.

The outer LSMR iteration uses this preconditioner to shape its search directions (Fong and Saunders 2011; Arridge et al. 2014; Yang et al. 2024). Once constructed, the same preconditioner can be reused to residualize the outcome and every covariate, and the algorithmic details are collected in Appendix A.

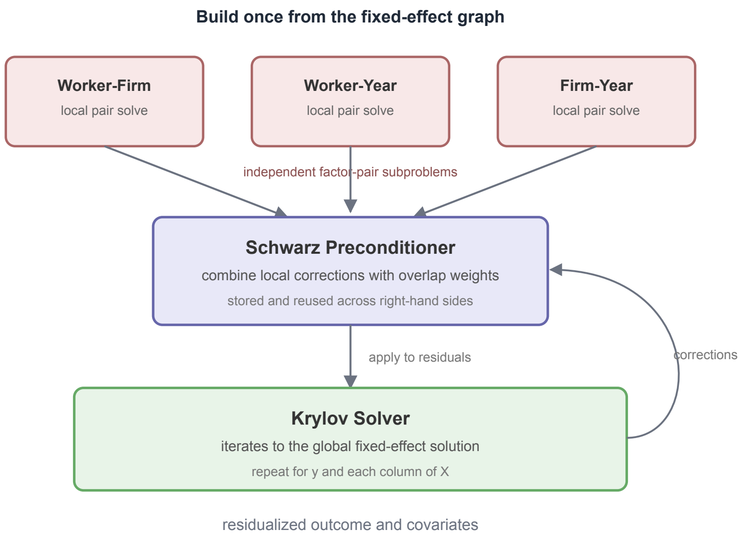

Figure 4: Summary of the factor-pair preconditioner. The fixed effects are split into overlapping factor pairs; each pair solve uses the Laplacian representation with approximate Cholesky; the resulting Schwarz preconditioner is then applied inside the outer LSMR iteration.

7. Benchmarks

The analysis above leads to a simple computational prediction: graph preconditioning should not dominate MAP in every setting, but it should be most useful when weak connectivity makes MAP's factor-by-factor demeaning pass information slowly through the fixed-effect graph. The benchmarks below test this prediction on controlled synthetic designs and standard public benchmark datasets.

The main runtime tables use package-level regression APIs, matching the public PyFixest benchmark suite, rather than isolated demeaning kernels. Each timing covers the full regression workflow: model setup, construction of the fixed-effect representation, residualization of the outcome and covariates, and estimation of the coefficient of interest. Separate tables report the memory cost of storing factor-pair information and verify that the preconditioned solver matches MAP to numerical tolerance.

The runtime tables also report the pairwise hardness statistic for the relevant factor pair and the observation share of the component attaining it. Smaller gaps indicate factor pairs on which MAP tends to converge more slowly; with three or more fixed effects, the statistic remains a pairwise diagnostic rather than a full convergence bound.

The benchmark section moves from controlled AKM designs to standard synthetic benchmarks and then to empirical datasets. The AKM designs vary mobility and sorting directly. The standard synthetic benchmarks use the simple/difficult DGPs from the fixest benchmark suite and synthetic datasets from the HDFE benchmark collection assembled by Sergio Correia. The empirical datasets from the same collection are included because synthetic DGPs can miss irregularities in real fixed-effect structures.

We benchmark four backends that separate MAP baselines from Krylov solvers with diagonal versus factor-pair preconditioning:

Backend	**Package**	**Algorithm**
`rust-map`	PyFixest	Vanilla Rust MAP, without acceleration.
`fixest`	R `fixest`	Accelerated MAP with Irons-Tuck and other optimizations (Bergé et al. 2026).
`FEM.jl`	`FixedEffectModels.jl`	Diagonally preconditioned LSMR (Fong and Saunders 2011; FixedEffectModels.jl Developers 2026).
`within`	PyFixest	LSMR with factor-pair Schwarz preconditioning.

Each CPU cell uses three benchmark trials run on an Apple M4 Mac mini with 10 CPU cores and 16 GB of memory running macOS 15.3.1. Times are medians over the trials that converged. When fewer than three converge, the table reports the successful-trial count; a failed cell means that all three trials reached the iteration cap.

We do not include `reghdfe` (Correia 2014; 2017) directly in the benchmark tables because Stata is not open source and we lack a license. `reghdfe` is a mature accelerated-MAP implementation; algorithmically, it belongs to the same family as `fixest`'s accelerated MAP.

7.1. Runtime Benchmarks

7.1.1. Controlled Synthetic Benchmarks: AKM Mobility and Sorting

We start with synthetic AKM-style panels, which let us vary one graph feature at a time while holding the rest of the data-generating process fixed. Changes in runtime then reflect the fixed-effect graph.

In the first synthetic design, we lower worker mobility across firms. As mobility falls, MAP still updates one fixed-effect dimension at a time, but fewer workers connect multiple firms, so information about firm effects moves more slowly through the iteration. The factor-pair preconditioner uses the worker-firm graph directly, so the preconditioned Krylov solver should become relatively more competitive in the low-mobility designs.

Mobility benchmark ($n = 1$M).

Scenario	**Gap (share)**	**`rust-map`**	**`fixest`**	**`FEM.jl`**	**`within`**
`akm_mobility_1`	1.38×10^{-1} (1.00)	0.325s	0.565s	0.467s	0.675s

Scenario	Gap (share)	`rust-map`	`fixest`	`FEM.jl`	`within`
`akm_mobility_2`	3.41×10^{-2} (1.00)	1.20s	0.996s	0.793s	0.502s
`akm_mobility_3`	5.18×10^{-3} (0.61)	13.5s	1.98s	2.00s	0.382s
`akm_mobility_4`	3.04×10^{-4} (0.27)	56.7s	6.25s	3.13s	0.351s
`akm_mobility_5`	1.65×10^{-3} (0.53)	73.5s	6.69s	3.29s	0.333s
`akm_mobility_6`	7.57×10^{-4} (0.37)	failed	7.13s	4.64s	0.329s

Note: AKM-style panel with 1M observations, one covariate, and worker, firm, and year fixed effects. Lower rows reduce worker mobility within a 10-period panel, thereby thinning the worker-firm graph. Lower mobility makes the worker-firm pair harder for MAP and correspondingly more favorable to the preconditioned method. A parenthesized fraction after a time is the number of converged trials; `failed` means none of the three trials converged. Gap is defined as $1 - \rho_{WF}$ for the worker-firm pair; parentheses report the observation share of the component attaining the gap.

The mobility benchmark matches the predicted pattern. When mobility is high, preconditioning yields little advantage, and `within` incurs setup costs that are not offset by improved convergence. As mobility declines, the worker-firm gap falls by more than two orders of magnitude, from 1.38×10^{-1} (1.00) to below 10^{-3} (the decline is not strictly monotone, because the reported gap is attained on different connected components), and MAP runtimes rise sharply. `within`'s runtime stays nearly flat across all designs. In the lowest-mobility configurations, the preconditioned method is the fastest backend because it operates on the worker-firm graph directly rather than propagating information through many sweeps that update one fixed-effect dimension at a time.

The second experiment increases sorting between workers and firms. Stronger sorting pushes the worker-firm graph toward weakly connected blocks: movers increasingly connect firms within the same group rather than linking different groups. MAP should therefore slow down again, because information about firm effects crosses groups only through a small number of movers. The preconditioned method should be less sensitive, since its factor-pair solves use the worker-firm graph directly.

Sorting benchmark ($n = 1$M).

Scenario	Gap (share)	`rust-map`	`fixest`	`FEM.jl`	`within`
`akm_sorting_1`	7.62×10^{-3} (0.83)	8.20s	1.57s	1.53s	0.374s
`akm_sorting_2`	2.41×10^{-3} (0.62)	11.9s	2.14s	2.20s	0.398s
`akm_sorting_3`	1.50×10^{-4} (0.59)	11.1s	2.17s	2.09s	0.402s
`akm_sorting_4`	2.14×10^{-5} (0.51)	15.7s	2.52s	2.15s	0.408s
`akm_sorting_5`	3.97×10^{-5} (0.53)	27.5s	2.99s	2.22s	0.400s

Note: AKM-style panel with 1M observations, one covariate, and worker, firm, and year fixed effects. Lower rows raise the degree of sorting among movers, pushing the worker-firm graph toward weakly connected blocks. Stronger sorting weakens cross-block information flow and thereby raises the value of factor-pair preconditioning. Gap is $1 - \rho_{WF}$ for the worker-firm pair; parentheses report the observation share of the component attaining the gap.

The sorting benchmark gives the parallel result. As movers sort more strongly across firms, the worker-firm graph separates into weakly connected blocks. The worker-firm gap falls by more than two orders of magnitude, and MAP-based runtimes rise. The gap does not fall monotonically, because different components attain the reported value in different rows, but its overall decline is clear. `within` remains nearly flat because its factor-pair preconditioner uses the worker-firm links directly, rather than relying on residual updates that cycle through one fixed-effect dimension at a time.

7.1.2. Standard Synthetic Benchmarks: fixest DGPs + Correia Synthetic

The AKM benchmarks varied mobility and sorting directly. We now turn to synthetic datasets that have become standard reference cases in fixed-effect software: they are public, easily reproducible, and already used to compare implementations.

The first family is the simple-versus-difficult benchmark data generating process from `fixest` (Bergé et al. 2026). Both designs use 10M observations, one covariate, and three fixed effects (worker, firm, year). The simple design has dense random mobility, whereas the difficult design has a sparse, nearly nested worker-firm structure. The simple design should be easy for MAP because its updates can pass information rapidly through a well-connected graph. The difficult design should be harder because worker and firm effects are nearly collinear. Factor-pair preconditioning should perform well on the difficult design, but may fail to amortize its setup cost on the simple design. For this comparison we add a fifth backend, `torch-cuda`: the PyFixest GPU backend, which runs the same diagonally preconditioned LSMR as `FEM.jl` on an NVIDIA CUDA device.

Simple vs. difficult design (10M observations, 3 FE).

Design	Gap (share)	`rust-map`	`fixest`	`FEM.jl`	`within`	`torch-cuda`
simple (dense graph)	8.57×10^{-1} (1.00)	1.96s	2.66s	2.26s	9.88s	4.73s
difficult (sparse graph)	1.67×10^{-5} (1.00)	322.2s	72.6s	27.7s	7.89s	8.73s

Note: Medians over three full regression calls. Both designs use 10M observations, one covariate, and three fixed effects. Gap denotes $1 - \rho_{WF}$ for the worker-firm pair, reported from the generated 1M version of the same DGP family; the runtime rows use the identical simple/difficult graph construction at 10M observations. `torch-cuda` denotes the PyFixest GPU LSMR backend with diagonal preconditioning (the same algorithm as `FixedEffectModels.jl`), run on an NVIDIA CUDA device; the local benchmark machine does not possess a CUDA GPU. The standalone `within` demeaning API decomposes one-shot runtime into reusable solver construction and batch solve: simple design 6.40s + 1.52s (75% setup); difficult design 0.782s + 0.955s (41% setup). PyFixest regression overhead is incurred in addition to these figures.

On the “simple” design, the graph is dense and the worker-firm gap is large. Both MAP backends therefore converge quickly, while `within` is slowest because the factor-pair preconditioner does not repay its setup cost. In the standalone demeaning breakdown, 75% of `within`’s demeaning time on the simple design is spent building the preconditioner.

On the difficult design, the ranking reverses. MAP convergence degrades sharply, to the point that `rust-map` without acceleration needs 72.6s. `within` completes in 7.89s, far faster than either MAP backend, because its factor-pair preconditioner captures the sparse worker-firm coupling directly. A nearly zero diagnostic gap identifies the regime in which MAP requires many sweeps to disentangle worker and firm effects. The setup share of the standalone demeaning time falls to 41%, indicating that the preconditioner setup is now amortized.

On the simple design, `torch-cuda` is not competitive with CPU `fixest`, since host-to-device transfer and kernel launch overheads outweigh the gain from parallelizing already inexpensive iterations. On the difficult design, GPU parallelism reduces runtime to 8.73s (3.2 times faster than CPU `FEM.jl`), but the iteration count remains governed by the quality of the diagonal preconditioner, which is limited on this graph. `within` at 7.89s remains faster on CPU by using a factor-pair preconditioner that captures the cross-factor coupling the diagonal cannot. Hardware acceleration and stronger preconditioning address different bottlenecks: the GPU speeds up each iteration, whereas on poorly conditioned designs the iteration count, which only the preconditioner controls, dominates runtime.

The second family of benchmarks comprises synthetic datasets drawn from the Correia HDFE benchmark collection. These datasets span a broader set of graph shapes: complete bipartite matching, uniform random matching with varying degrees of connectivity, assortative matching, and a small path-like design. They provide an independent public reference set for comparing the same software backends outside the AKM generator.

Correia synthetic benchmarks.

Dataset	Gap (share)	`rust-map`	`fixest`	`FEM.jl`	`within`
`synthetic-complete`	1.000 (1.00)	0.078s	0.066s	0.044s	0.150s
`synthetic-uniform-easy`	0.651 (1.00)	0.129s	0.147s	0.106s	0.135s
`synthetic-uniform-hard`	0.184 (1.00)	0.320s	0.367s	0.340s	1.05s
`synthetic-uniform-harder`	0.0249 (1.00)	0.939s	1.29s	0.551s	0.592s
`synthetic-assortative`	0.00133 (0.70)	22.4s	1.83s	2.35s	0.416s

Note: Medians over three runs. Gap denotes $1-\rho$ for the `id1-id2` pair after the same singleton pruning; parentheses report the observation share of the component attaining the gap. Smaller gaps correspond to slower two-way MAP geometry. The `synthetic-zigzag` dataset is omitted because `rust-map` reaches its iteration cap while the converged runtimes of the other backends span several orders of magnitude. It is more useful as a stress case than as a compact runtime comparison.

These results are less clear than the controlled AKM experiments, which is itself informative. Several datasets are small enough, or sufficiently well connected, that setup cost matters as much as conditioning; on the complete and easier uniform designs, the gap is large and low-overhead methods perform well. The harder rows reverse the ranking. By `synthetic-uniform-harder`, the gap has fallen to 0.0249 (1.00) and `within` has become competitive with the fastest backend. On the assortative benchmark, the preconditioned method exhibits its clearest advantage: sorting generates cross-factor structure that MAP's updates handle only slowly.

7.1.3. Standard Real-Data Benchmarks: Correia Collection

We next turn to real benchmark data from the Correia collection. Synthetic data sets match the overall shape of empirical co-occurrence graphs but smooth away the irregularities that often drive runtime: a few units that appear far more often than the rest, thin connections between otherwise dense groups, many small disconnected pieces, and interactions between identifiers that go beyond a single two-way pair. Real data carries these features by default, and therefore tests the solvers in conditions that controlled DGPs only approximate.

Correia real-data benchmarks.

Dataset	Gap (share)	`rust-map`	`fixest`	`FEM.jl`	`within`
`credit`	0.402 (1.00)	0.191s	0.140s	0.139s	0.176s
`soccer`	0.889 (1.00)	0.018s	0.017s	0.014s	0.043s
`enron`	0.00704 (0.98)	4.12s	0.583s	0.525s	0.433s
`github`	0.000630 (0.32)	84.2s	1.95s	3.48s	0.285s
`patents`	0.000518 (0.95)	26.7s	1.59s	1.12s	0.385s
`workers`	0.000274 (0.63)	133.5s	3.98s	3.77s	0.199s
`schools`	0.00221 (1.00)	10.00s	1.24s	1.01s	0.192s
`directors`	0.000512 (0.30)	12.4s	0.383s	0.987s	0.293s

Note: Medians over three runs on singleton-dropped samples, as produced by the PyFixest benchmark suite. The gap is $1 - \rho$ for the `id1-id2` pair after the same singleton pruning; parentheses report the observation share of the component attaining the gap. A small gap with a limited component share, as in `directors`, indicates a hard subcomponent but not necessarily whole-sample MAP difficulty.

The empirical datasets show the same pattern in less stylized form. Accelerated MAP is difficult to outperform on small or compact graphs such as `credit` and `soccer`, where the gaps are large. The `directors` row illustrates why the component share is useful: the worst component is hard, but it contains 30% of the observations, so the full problem is not as costly for MAP as the gap alone would suggest. On larger networks with hard components covering a substantial portion of the sample, particularly `enron`, `github`, `patents`, `workers`, and `schools`, the factor-pair preconditioner is fastest by a wide margin.

7.2. Poisson / PPML Benchmark

Generalized linear models with high-dimensional fixed effects are typically estimated by iteratively reweighted least squares (IRLS). Each IRLS step fits a weighted least squares problem in which the response and covariates are demeaned against the fixed effects, with weights that are updated between iterations (Correia et al. 2020; Stammann 2018). The demeaning operation is identical to the process described in the prior sections. Any acceleration of fixed-effect demeaning therefore propagates into the GLM runtime, multiplied by the number of IRLS iterations.

7.2.1. Preconditioner reuse across IRLS

The IRLS structure also enlarges the window over which a preconditioner setup cost can amortize. A factor-pair preconditioner depends on the fixed-effect graph (invariant across IRLS iterations) and on the IRLS weights (which do change between iterations). If the weights do not move much, a slightly stale preconditioner is still effective. We exploit this property by building the preconditioner once and reusing the stale version on subsequent IRLS iterations. Staleness slows the outer Krylov solver but does not bias its solution; the iteration still converges to the correct demeaned residuals. The construction cost is therefore paid once per regression rather than once per IRLS step.

We benchmark this strategy on the simple-versus-difficult DGPs from the `fixest` benchmark suite (Bergé et al. 2026) at $n = 1\text{M}$ observations, one covariate, and two or three fixed effects (worker-year, or worker-firm-year), using the same iteration protocol as the OLS benchmarks. The compared backends are R `fixest`'s `fepois`, `GLFixedEffectModels.jl`, and two PyFixest `fepois` backends: the default unpreconditioned `rust-map` and the preconditioned `within` solver.

Poisson benchmarks (1M observations, one covariate).

Design	FE	`fixest`	`rust-map`	`GLFEM.jl`	`within`
simple (dense graph)	2	1.51s	1.81s	2.21s	3.79s
difficult (sparse graph)	2	1.47s	1.76s	2.22s	3.73s
simple (dense graph)	3	6.66s	10.6s	8.70s	8.65s
difficult (sparse graph)	3	337.7s	failed	180.0s	6.24s

Note: Medians over three full IRLS regression calls at $n = 1\text{M}$ and one covariate. `fixest` is R `fixest::fepois`; `rust-map` and `within` are the PyFixest `fepois` routine with the unpreconditioned MAP backend and the factor-pair preconditioned solver, respectively; `GLFEM.jl` is `GLFixedEffectModels.jl`. `failed` indicates that all three `rust-map` trials reached the 10000-iteration MAP cap without converging.

The patterns observed in the OLS benchmarks carry over. With two fixed effects, all four backends complete in comparable time on both designs and R `fixest` is fastest: the worker-year pair alone does not create a regime in which the preconditioner amortizes its setup cost, even with the IRLS-induced reuse. With a third fixed effect, the picture changes. On the three-FE simple design MAP still converges and `fixest` remains fastest, but the unaccelerated `rust-map` slows to 10.6s, `GLFEM.jl` to 8.70s, and `within` lands between them at 8.65s. The three-FE difficult design provides the sharpest contrast: `rust-map` does not converge within the iteration cap, `GLFEM.jl` takes 180.0s, `fixest`'s IRLS loop takes several minutes with large run-to-run variance, and `within` finishes in 6.24s, 54 times faster than `fixest` and 29 times faster than `GLFEM.jl`. The factor-pair preconditioner captures the same sparse worker-firm coupling that drove the OLS benchmark results, and the IRLS outer loop inherits the gain without modification.

7.3. Memory Use

MAP uses little memory: a sweep needs only the current residuals and per-level group sums. A factor-pair preconditioner, by contrast, must retain the pair structure between iterations. A practically relevant question is therefore how much more memory the preconditioned Krylov solver requires relative to MAP. We measure peak resident set size (peak RSS), the largest quantity of physical memory consumed by the process during a run, on the simple and difficult fixed-effect DGPs. These serve as representative probes rather than special memory cases: the dominant storage terms are the data matrix, the fixed-effect encodings, and the reusable factor-pair objects, so the results should largely generalize across datasets of comparable dimensions. We do not use this section to compare against `fixest` or `FixedEffectModels.jl`, because cross-language peak RSS also reflects R and Julia runtime overhead, data-loading choices, garbage collection, and package internals. The diagnostic of interest is the incremental memory cost of substituting the preconditioned Rust backend for Rust MAP within the same Python package. Because the surrounding regression code is shared, this comparison isolates the difference attributable to the demeaning strategy. Both backends are executed in isolated processes and report peak RSS via `ru_maxrss`.

Memory footprint (3 FE, one covariate).

Design	Gap (share)	`rust-map`	`within`
100K observations			
simple (dense graph)	8.57×10^{-1} (1.00)	307 MB	363 MB
difficult (sparse graph)	1.30×10^{-3} (1.00)	309 MB	356 MB
1M observations			
simple (dense graph)	8.57×10^{-1} (1.00)	660 MB	992 MB
difficult (sparse graph)	1.67×10^{-5} (1.00)	648 MB	835 MB

Note: Peak RSS denotes peak resident set size, measured from isolated Python processes. One covariate, three fixed effects. These two DGPs serve as representative probes; we do not claim that memory behavior is design-specific. Gap denotes $1 - \rho_{WF}$ for the worker-firm pair in the corresponding generated design.

At 100K observations, the preconditioner adds 47–56 MB. At 1M observations the overhead is larger in absolute terms (187–332 MB), but it remains modest relative to the data footprint of a panel with one covariate. The preconditioned solver consumes more memory than MAP, yet the additional storage for factor-pair co-occurrences, partition weights, and local approximate Cholesky factors remains small relative to the full regression problem.

7.4. Numerical Equivalence

Before trusting a new fixed-effect solver, we must verify that it reproduces the estimates of existing routines. MAP and LSMR-style routines differ in their convergence checks, residual norms, stopping thresholds, and iteration caps, so two correct implementations can return coefficients that agree only up to their tolerances.

We use the 100K-observation simple and difficult data generating processes from the fixest benchmarks as diagnostic cases. The simple design examines the "easy-to-converge" regime where all methods should agree almost exactly. The difficult design examines the regime where conditioning is poor and small differences in stopping rules are most likely to emerge. The graph diagnostic distinguishes these cases: the worker-firm gap is large in the simple design and near zero in the difficult design. The table below collects one regression coefficient for all four backends.

Coefficient agreement (100K observations, 3 FE, one covariate).

Design	Backend	$\hat{\beta}_1$	Avg |diff|	Max |diff|
table.cell(rowspan: 4) [simple]	`rust-map`	0.99871660	–	–
	`within`	0.99871660	2.0×10^{-12}	2.0×10^{-12}
	`fixest`	0.99870000	1.7×10^{-5}	1.7×10^{-5}
	`FEM.jl`	0.99871660	9.2×10^{-14}	9.2×10^{-14}
table.cell(rowspan: 4) [difficult]	`rust-map`	1.00321070	–	–
	`within`	1.00321085	1.5×10^{-7}	1.5×10^{-7}
	`fixest`	1.00320000	1.1×10^{-5}	1.1×10^{-5}
	`FEM.jl`	1.00321069	6.7×10^{-9}	6.7×10^{-9}

Note: $\hat{\beta}_1$ is the slope coefficient on `x1`. Differences are absolute slope-coefficient deviations from `rust-map`, averaged (Avg) and maximized (Max) across the reported coefficient; the two are identical in this one-covariate benchmark. `within` is the PyFixest preconditioned Rust backend.

The `within` and `FEM.jl` rows are almost identical to `rust-map` at the reported defaults. R `fixest` is also close; its largest reported coefficient difference is 1.7×10^{-5}. The remaining differences reflect the packages' stopping rules and numerical tolerances.

8. Software

The solver studied in this paper is available as open-source software through the `within` project (within Developers 2026). The computational core is implemented in Rust and exposed through Rust, Python, and R interfaces; each language binding invokes the same underlying solver. This shared core matters for reproducibility and adoption: the same algorithm can be called from Rust, Python, or R without reimplementation.

Interface	Package	Registry	Install
Rust	`within`	crates.io	`cargo add within`
Python	`within-py`	PyPI	`pip install within-py`
R	`withinr`	py-econometrics R-universe	`install.packages("withinr", repos = "https://py-econometrics.r-universe.dev")`

The Rust crate exposes the lower-level solver together with its configuration types. The Python and R packages provide solver-level `solve` and `solve_batch` APIs for residualizing one or several right-hand sides. The algorithm is also available as a demeaning backend for PyFixest (PyFixest Developers 2026).

Here is a minimal Python example, in which we demean an outcome variable and two covariates against worker and firm fixed effects, before we run the FWL fit on the demeaned variables:

```python
import numpy as np
from within import solve_batch

# Worker and firm identifiers as a column-major uint32 array
n = 100_000
categories = np.asfortranarray(np.column_stack([
    np.random.randint(0, 5_000, n).astype(np.uint32),
    np.random.randint(0,   500, n).astype(np.uint32),
]))

# Outcome and covariates
beta = np.array([1.0, -2.0])
X = np.random.randn(n, 2)
y = X @ beta + np.random.randn(n)

# Residualize y and X jointly; the preconditioner is reused across columns
res = solve_batch(categories, np.column_stack([y, X]))
y_tilde, X_tilde = res.demeaned[:, 0], res.demeaned[:, 1:]

# FWL on the demeaned variables
beta_hat = np.linalg.lstsq(X_tilde, y_tilde, rcond=None)[0]
```

9. Conclusion

We have developed a graph-based preconditioner for fixed-effect demeaning and compared it with MAP on synthetic and empirical benchmarks. Which of the two solvers is faster depends on the structure of the fixed effects graph.

When the graph is dense and well connected, MAP is difficult to outperform. Its sweeps are cheap, and a preconditioner mostly adds overhead, as in the simple fixest design and the smaller, well-connected Correia datasets. When a factor pair is sparsely connected, through low mobility, strong sorting, or near-nesting, MAP passes information across the graph slowly, and the factor-pair preconditioner is faster, often by a wide margin.

The preconditioner depends only on the fixed-effect graph, so it can be built once and reused across demeaning calls. Reuse matters most when a single estimation issues many such calls: IRLS-based GLMs such as PPML demean once per iteration, so the construction cost is paid once while the faster convergence can accrue at every iteration step. In our PPML benchmark on a hard three-way design, `within` finishes in seconds where the MAP-based routines take minutes or fail to converge.

Which solver to prefer depends on the fixed-effect graph. Accelerated MAP and diagonally preconditioned LSMR are good defaults across much of the range our benchmarks cover: they carry (almost) no setup cost, and on dense, well-connected graphs they are the fastest options. The factor-pair preconditioner amortizes its setup cost in the remaining cases, and we recommend it when the gap diagnostic is small, when a fit is unexpectedly slow, or for IRLS-based GLM, which repeat the demeaning step many times.

Appendix A: Factor-Pair Schwarz Algorithm

Algorithm 1 gives the implementation corresponding to the construction summarized in Figure 4.

Algorithm 1. Factor-Pair Schwarz Preconditioner

Inputs

- Observation-level factor codes for Q absorbed dimensions.
- Diagonal weights W and a local solver configuration.
- Krylov residual r in coefficient space.

Preconditioner setup

- Enumerate all unordered factor pairs (q, r) with $q < r$.
- For each pair, build weighted count blocks G_{qq}, G_{rr} and the weighted cross-tabulation C_{qr}.
- Split the induced bipartite graph into connected components and create one Schwarz subdomain s per component.
- If fixed-effect level j appears in c_j subdomains, store the partition weight $\omega_j = \frac{1}{\sqrt{c_j}}$.
- For each subdomain, sign-flip one side to obtain a local Laplacian, project the local right-hand side off the component constant, and build a zero-mean local solve.
- Use a Schur-complement local solver: small reduced systems are solved directly, while larger reduced symmetric diagonally dominant (SDD) or Laplacian systems are solved with randomized approximate Cholesky.

Krylov application

- Initialize $z = 0$.
- For each subdomain s, form $h_s = \tilde{D}_s R_s r$.
- Compute the approximate local correction $u_s \approx A_s h_s$ on the normalized subspace.
- Accumulate $z \leftarrow z + R_{s'} \tilde{D}_s u_s$.
- Return $z = M^{-1} r$.

References

Abowd, John M., Francis Kramarz, and David N. Margolis. 1999. "High Wage Workers and High Wage Firms." *Econometrica* 67 (2): 251–333. https://doi.org/10.1111/1468-0262.00020.

Arridge, Simon R., Marta M. Betcke, and Lauri Harhanen. 2014. "Iterated Preconditioned LSQR Method for Inverse Problems on Unstructured Grids." *Inverse Problems* 30 (7): 75009. https://doi.org/10.1088/0266-5611/30/7/075009.

Berge, Laurent. 2018. *Efficient Estimation of Maximum Likelihood Models with Multiple Fixed-Effects: The R Package Fenmlm.* Technical Report Nos. 2018–13.

Bergé, Laurent R., Kyle Butts, and Grant McDermott. 2026. "Fixest: A Fast and Feature-Rich Framework for Econometric Estimations in R.". https://doi.org/10.48550/arXiv.2601.21749.

Correia, Sergio. 2014. "Reghdfe: Stata Module to Perform Linear or Instrumental-Variable Regression Absorbing Any Number of High-Dimensional Fixed Effects.". https://github.com/sergiocorreia/reghdfe.

Correia, Sergio. 2017. *Linear Models with High-Dimensional Fixed Effects: An Efficient and Feasible Estimator.* Technical report. https://hdl.handle.net/10.2139/ssrn.3129010.

Correia, Sergio, Paulo Guimarães, and Tom Zylkin. 2020. "Fast Poisson Estimation with High-Dimensional Fixed Effects." *The Stata Journal* 20 (1): 95–115. https://doi.org/10.1177/1536867X20909691.

FixedEffectModels.jl Developers. 2026. "Fixedeffectmodels.jl: Fast Estimation of Linear Models with High Dimensional Categorical Variables.". https://github.com/FixedEffects/FixedEffectModels.jl.

Fong, David Chin-Lung, and Michael Saunders. 2011. "LSMR: An Iterative Algorithm for Sparse Least-Squares Problems." *SIAM Journal on Scientific Computing* 33 (5): 2950–71. https://doi.org/10.1137/10079687X.

Frisch, Ragnar, and Frederick V. Waugh. 1933. "Partial Time Regressions as Compared with Individual Trends." *Econometrica* 1 (4): 387–401. https://doi.org/10.2307/1907330.

Gao, Yuan, Rasmus Kyng, and Daniel A. Spielman. 2025. "AC(k): Robust Solution of Laplacian Equations by Randomized Approximate Cholesky Factorization." *SIAM Journal on Scientific Computing*,. https://rasmuskyng.com/papers/GKS25.pdf.

Gaure, Simen. 2013. "OLS with Multiple High Dimensional Category Variables." *Computational Statistics & Data Analysis* 66 : 8–18. https://doi.org/10.1016/j.csda.2013.03.024.

Goldsmith-Pinkham, Paul. 2026. *Tracking the Credibility Revolution across Fields.* Technical report.

Guimaraes, Paulo, and Pedro Portugal. 2010. "A Simple Feasible Procedure to Fit Models with High-Dimensional Fixed Effects." *The Stata Journal* 10 (4): 628–49. https://doi.org/10.1177/1536867X1101000406.

Irons, Bruce M., and Richard C. Tuck. 1969. "A Version of the Aitken Accelerator for Computer Iteration." *International Journal for Numerical Methods in Engineering* 1 (3): 275–77. https://doi.org/10.1002/nme.1620010306.

Lovell, Michael C. 1963. "Seasonal Adjustment of Economic Time Series and Multiple Regression Analysis." *Journal of the American Statistical Association* 58 (304): 993–1010. https://doi.org/10.1080/01621459.1963.10480682.

PyFixest Developers. 2026. "Pyfixest: Fast High-Dimensional Fixed Effects Regression in Python.". https://github.com/py-econometrics/pyfixest.

Spielman, Daniel A., and Shang-Hua Teng. 2014. "Nearly Linear Time Algorithms for Preconditioning and Solving Symmetric, Diagonally Dominant Linear Systems." *SIAM Journal on Matrix Analysis and Applications* 35 (3): 835–85. https://doi.org/10.1137/090771430.

Stammann, Amrei. 2018. "Fast and Feasible Estimation of Generalized Linear Models with High-Dimensional K-Way Fixed Effects.". https://doi.org/10.48550/arXiv.1707.01815.

Toselli, Andrea, and Olof Widlund. 2005. *Domain Decomposition Methods: Algorithms and Theory*. Springer. https://doi.org/10.1007/b137868.

within Developers. 2026. "Within: High-Performance Fixed-Effects Solver for Econometric Panel Data.". https://github.com/py-econometrics/within.

Xu, Jinchao. 1992. "Iterative Methods by Space Decomposition and Subspace Correction." *SIAM Review* 34 (4): 581–613. https://doi.org/10.1137/1034116.

Yang, Mei, Gul Karaduman, and Ren-Cang Li. 2024. "Flexible Modified LSMR for Least Squares Problems.".